Final Report

CS 4850 – Fall 2025

ZX-1 AISmartSensing

Cole Myers, Maddox McDaniel, Jack Drellos

Sharon Perry 12/8/25

Website:

https://ai-for-sensing-ksu2025.github.io/AI-For-Sensing/

GitHub:

https://github.com/AI-For-Sensing-KSU2025/AI-For-Sensing

Stats:

LOC: 235

Project Components: 1

Total Man Hours: 435

Completion Percentage: 95%

Table of Contents

Introduction

This is the final report for our project. We are using the UpPoinTr AI model Pipeline to recreate 3d environments from radar point cloud data from the ColoRadar dataset. We are adjusting this model to use spiking neural networks in certain modules of the pipeline in order to increase the energy efficiency of the model when inferenced. To achieve this, we are using SpikingJelly, a library which adds spiking implementations to PyTorch, a popular python deep learning package.

Requirements

1.0 Introduction

1.1 Overview

This document is the Software Requirements Specification for development of our AI for Smart Sensing Software. The following sections will list specifications, including but not limited to; requirements, both functional and non-functional, as well as constraints for the software.

1.2 Project Goals

A working model that transforms radar signals into 3D point clouds;

Evaluation comparing SNN with baseline approaches;

Visualizations of environmental maps reconstructed from radar data.

1.3 Definitions and Acronyms

SNN – Spiking Neural Network

mmWave – mmWave is short for millimeter wave and refers to high frequency radio waves which can be used for remote sensing.

PyTorch – Open-source deep learning framework

Point clouds – A discrete set of data points which can be used to represent a 3D object. For our use point clouds are used to represent the environment of the room being surveyed.

ColoRadar – A publicly available mm-Wave Radar dataset recorded and published by the Autonomous Robotics and Perception Group at the University of Colorado Boulder.

Lidar – An acronym for laser imaging, detection, and ranging. It is a method for mapping an object/environment.

1.4 Assumptions

It is assumed that the user's machine will be sufficiently powerful to run the software as specified in the hardware requirements. Therefore, any time-based benchmarks are dependent on the user's machine meeting at least the minimum hardware requirements.

It is assumed that a user will have access to their own methods of obtaining mmWave data.

2.0 Design Constraints

2.1 Environment

Our system will exist in its own Python virtual environment. This environment will be running Python 3.11 and will utilize the PyTorch library.

2.2 User Characteristics

Currently, users are expected to be familiar with mmWave technology and deep learning frameworks. The product is intended to be for these users but in the future this technology would be used for more mainstream applications. A few examples of these applications would be, smart homes, assisted living, and security monitoring systems. These applications would require the product be simplified for the general public.

2.3 System Hardware Requirements

Any machine learning model has intensive hardware requirements and our system is no different. Our minimum recommended hardware is as follows; a multi-core processer such as an Intel Core i7, 64GB of RAM, a GPU with at least 12GB of VRAM such as NVIDIA GeForce RTX 3060. Our general recommended hardware is as follows; Intel Core i9, 128GB of RAM, multiple GPUs with at least 12 GB of VRAM.

3.0 Functional Requirements – EXAMPLE for MOBILE APP

3.1 Data Input

The system shall be able to process any user input of valid mmWave data. mmWave data is generally stored as binary files so that is what the system will be primarily built for.

3.2 Data Visualization

The system shall be able to use mmWave data to make 3D visualizations of the surrounding environment.

3.3 Advanced Use Cases

The system should have an interactive tool which can be used to visualize the mmWave data and generated point clouds. The interactive tool is a major positive but not a necessary element.

4.0 Non-Functional Requirements (use if applicable)

4.1 Usability

The system must be user-friendly and efficient. Our goal is for a user to be able to simulate an environment in no more than 30 seconds.

4.2 Scalability

The system will be designed with the intention of potential future upwards scalability. Currently, the system is intended for small indoor rooms but future applications could be used in larger rooms or outdoor areas.

Design

1. Introduction and Overview

This document is the Software Design Document (SDD) for our AI for Smart Sensing Software. Subsequent sections will document many of the key design features of our software.

2. Design Considerations

2.1 Assumptions and Dependencies

It is assumed that a user will have access to a machine with either Windows or Linux operating systems. Our system will be developed on Windows OS but it will be coded almost entirely in Python so it will have cross-platform operability for Linux.

It is assumed that a user will have access to a machine with, at the very least, our listed minimum required hardware which is as follows; a multi-core processer such as an Intel Core i7, 64GB of RAM, a GPU with at least 12GB of VRAM such as NVIDIA GeForce RTX 3060. Our general recommended hardware is as follows; Intel Core i9, 128GB of RAM, multiple GPUs with at least 12 GB of VRAM.

It is assumed that a user will be familiar with mmWave radar data and will be able to supply their own data into the software. In the future this work will be used in systems for the general public so it will be designed to be interoperable with mmWave sensors but at its current stage the data needs to be supplied by the user.

2.2 General Constraints

2.2.1 Hardware Constraints

Due to the nature of AI models, one of our main constraints is the hardware itself. We need very modern and intensive hardware and even just obtaining our minimum requirements is no easy task.

2.2.2 Real-World Constraints

Our system is built to create point clouds for smaller indoor environments only. These environments are the easiest to process and generate. This mainly effects are estimation of hardware requirements and processing time.

2.2.3 Dataset Constraints

Currently we are constrained to using the ColoRadar dataset. This dataset contains files which are very large, it also only has 10 indoor files that we can actually use for our purposes.

2.3 Development Methods

We will utilize Agile methodologies for the development of our software. We will follow the common Agile cycle of analyzation, planning, design, building, testing and review. We will split the work into 3 separate sprints each lasting 4 weeks. For testing in particular we also plan to utilize pair testing. We believe that four eyes are always better than two and it is very important to have an observer there for testing to find any blind spot the main programmer may have.

3. Architectural Strategies

We decided to code our software in Python 3.11. Python was an easy choice because it has cross-platform compatibility and a wide array of popular libraries that can be used for our software. The main library which is being used is the library, PyTorch. This library is an open-source deep learning framework which will allow us to create the SNN we need. Currently the program will operate entirely on one machine and does not have to interface with any other software or hardware. In the future, the program will interface with mmWave sensors themselves so they can receive live mmWave data. It will also interface with some other app manufactured for the general public to easily observe the 3D visualizations.

4. System Architecture

The architecture will have three parts:

1. The ColoRadar dataset
2. Model creation, training and Validation
3. Model conversion from artificial neural network to spiking neural network

The ColoRadar dataset has the necessary data to train and validate our models. It has both radar data, which is what we are training our model on, and lidar data which we will use as our ground truth. We will create a convolution neural network and train it on our dataset. Once we have a working model, we will then convert the networking into a spiking neural network made easy through PyTorch.

5. Detailed System Design

5.1.1 Classification

Our dataset: ColoRadar

5.1.2 Definition

This is the dataset containing radar and lidar data which we will use to train our model.

5.1.3 Constraints

While this dataset has high quality assets, its sample size is quite small. We may find that there are too few samples to adequately train our model.

5.1.4 Resources

The storage needed for the dataset is quite large. At least 500 gigabytes. We will have to load in samples on at a time for training. We will have one storage drive containing the dataset, the system will then load individual samples into the project to train/test and when it is finished the data will be removed. This will keep the storage size of our project small even though our samples are quite large.

5.1.4 Definition

The storage needed for the dataset is quite large. At least 500 gigabytes. We will have to load in samples on at a time for training. We will have one storage drive containing the dataset, the system will then load individual samples into the project to train/test and when it is finished the data will be removed. This will keep the storage size of our project small even though our samples are quite large.

5.1.4 Interface/Exports

The dataset simply holds all the data we will use to train and test the model.

5.2.1 Classification

Subsystem: Model creation and training.

5.2.2 Definition

This system is responsible for declaring our model type and training it. Once the model is trained it will then test the model's accuracy.

5.2.3 Constraints

Our model will most likely need to be quite large to learn the intricacies of the data. This large model combined with the large size of our samples means that training will take a very long time. We will be limited on how large we can make the model to keep it from training for weeks on end.

5.2.4 Resources

Time is the biggest resource taken by this system. Training takes lots of time and this model will take a long time to train. We will need to be weary of this when adjusting the models hyperparameters.

5.2.5 Interface/Exports

This subsystem should output a model capable of accurately creating digital 3D models in the form of point clouds when given radar data.

6 Glossary

CNN – Convolutional Neural Network

SNN – Spiking Neural Network

mmWave – mmWave is short for millimeter wave and refers to high frequency radio waves which can be used for remote sensing.

PyTorch – Open-source deep learning framework

Point clouds – A discrete set of data points which can be used to represent a 3D object. For our use point clouds are used to represent the environment of the room being surveyed.

ColoRadar – A publicly available mm-Wave Radar dataset recorded and published by the Autonomous Robotics and Perception Group at the University of Colorado Boulder.

Lidar – An acronym for laser imaging, detection, and ranging. It is a method for mapping an object/environment.

Technical Details and Meeting Transcripts

3D Representation of an Internal Environment

A 3D representation of an internal is achieved by using sensors to capture data of an indoor space to create a 3D Point Cloud of the environment. For our purposes we feed mmWave radar data through our AI model to create these point clouds.

3D Point Cloud

A 3D point cloud is a representation of a discrete set of data points in space. Each individual point has it's own unique set of Cartesian coordinates which are used to determine it's position in space. We use radar data and feed it through our model to generate point clouds of an environment.

mmWave Radar Data

mmWave radar data is a specific frequency of radio waves which we can use for remote sensing. The major advantage of mmWave sensors is that they are more economical than other mainstream methods such as LiDAR. This makes it much more realistic for widespread use as an indoor sensor.

ColoRadar

ColoRadar is a publicly available mm-Wave Radar dataset recorded and published by the Autonomous Robotics and Perception Group at the University of Colorado Boulder. We rely on their dataset for training our model. They also have an official GitHub repository which has been very valuable for developing our model.

PyTorch

PyTorch is an open-source deep learning framework we use for building and training our AI model.

SNN (Spiking Neural Network)

A spiking neural network is a type of ANN (artificial neural network) that works by attempting to mimic how neurons in the brain work. They process spikes overtime and use the timing of those spikes to encode information. Spiking neural networks are important for our purposes because due to the fact that they're more computationally efficient than other neural networks. They don't process data continuously and instead only do so based on these spikes. This makes our model much more suited to be installed in small sensor devices that don't have much processing power.

UpPointTr

UpPoinTr is a GitHub project by the authors Mopidevi, Ajay Narasimha and Harlow, Kyle and Heckman, Christoffer. This was a project by some of the team members of the original ColoRadar project to create their own model. We plan to use their work to help create our SNN model.

Spiking Jelly

Spiking Jelly is a python library containing tools to create spiking neural networks through PyTorch

8. Database Connection

Any project using an AI model needs access to a large amount of training data. Our project is no different, and we use the publicly available ColoRadar dataset from the ColoRadar project performed at the University of Colorado Boulder. Unfortunately, there is an issue with their database library which is causing pulls to be denied. This makes it impossible for us to set up a server for our dataset, which is quite large. We contacted one of the students on the project, Anna Zavei-Boroda who agreed to help us. She is trying to fix this issue and said she will notify us when it is fixed.

For now, we are proceeding without the database and have downloaded the entire dataset. The UpPoinTr model that our research is adapting into a spiking neural network was built using the ColoRadar dataset, so little to no modifications will be needed to connect the database to the model.

This is the official ColoRadar library:

https://github.com/arpg/coloradar-library

The dataset can be downloaded from the following SharePoint:

https://o365coloradoedu-my.sharepoint.com/personal/chhe5305_colorado_edu/_layouts/15/onedrive.aspx?id=%2Fpersonal%2Fchhe5305%5Fcolorado%5Fedu%2FDocuments%2FColoRadar&ga=1&LOF=1

9. Project Reconstruction

- First, verify that you have the necessary prerequisites, both hardware and software.
- Your OS, we used Linux Ubuntu 24.04 for its convenience in the bash interface, it's CUDA toolkit support, and reliable machine learning package ecosystem. This is doable on other non-Unix/Linux based systems, but you would have to use WSL2. GPU with CUDA support (NVIDIA GPU of course) - make sure your drivers are installed correctly Python 3.7+ CUDA 9.0+ installed (don't use v13.0 (latest release) as we had problems when testing with it) GCC 4.9+ compiler Git installed Conda or virtualenv for environment management
- Next is to clone the UpPoinTr Repository.
- Create your virtual environment, ideally with Conda, and install all other dependencies, such as PyTorch, timm, open3d, tensorboardX, matplotlib, tqdm, and h5py. All of these are listed in the requirements.txt of the now cloned UpPoinTr repository. Make sure to also install the PointNet++ repo, "git+https://github.com/erikwijmans/Pointnet2_PyTorch.git#egg=pointnet2_ops&subdirectory=pointnet2_ops_lib", as well as the KNN_Cuda repo, https://github.com/unlimblue/KNN_CUDA/releases/download/0.2/KNN_CUDA-0.2-py3-none-any.whl.
- From here, it is an excellent idea to make sure that all of your dependencies are installed, by verifying your imports.
- Next, verify that your data has been downloaded. The repo comes with some data preinstalled in the 'data' folder, but if you would like to expand that, you may download off of the ColoRadar dataset as previously listed.
- From there, you have the option to train your own model, or instead use one that is pretrained, either by us or by the original repo creators. Keep in mind, that this will be dependent upon your own system's hardware and functionality. Make a choice that is realistic for your machine's capabilities. Training a model from the ground up on a larger data set will be much more expensive than using a pre-trained model on the smaller, included dataset.
- After choosing your model, choosing your dataset, and training the model. To replicate our project, we have decided to convert the Neural Network into a Spiking Neural Network using our own scripts.
- Running this altogether will provide the grounds for you to start your research.

Project Plan

1.0 Project Overview / Abstract (Research)

This project focuses on Artificial Intelligence for sensor data processing and environmental perception. Students will work with mmWave radar data collected from indoor environments and develop models to generate 3D point clouds that capture the structure of the surroundings.

Radar signals are inherently sparse, noisy, and low-resolution, making them challenging to interpret using traditional methods. This project explores the use of spiking neural networks (SNNs)—a brain-inspired class of AI models—to transform radar signals into meaningful spatial representations. Students will learn how to encode radar data into spike-based formats and train SNNs to reconstruct accurate 3D maps.

The outcome will be an AI-based sensing system that performs environmental mapping using non-visual, privacy-preserving radar signals, suitable for applications like smart homes, assisted living, and security monitoring.

The previous paragraphs were from our project overview on our project's entry in the project list. We do not have much to add yet as we have still not had the opportunity to meet with Dr. Xie.

2.0 Project website

Our GitHub link is, https://github.com/AI-For-Sensing-KSU2025, we will eventually launch an actual website on GitHub Pages as well.

Deliverables - Specific to Your Project

AI-Based Sensing System (Details on hardware to be specified after meeting with Dr. Xie) (Group Assignment)

Weekly Activity Reports (WARs – Individual Assignment)

Team Status Report (TSR – Group Assignment)

Peer Reviews (Individual Assignment)

Project Plan (Group Assignment)

SRS, SDD, STP & Dev Doc (Group Assignment)

Prototype Presentation for Peer Review (Group Assignment)

C-Day Application/Submission (Group Assignment)

Possible Research Paper if results are publishable (Group Assignment)

Final Report Package: Final Report, Source code and website on our GitHub page, Final Presentation (Group Assignment)

Group Meeting Schedule Date/Time

Student Team Availability

X	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
Cole	Before 10 After 8	Open	Open	Open	Before 10 After 8	Before 10 After 8	Before 10 After 8
Maddox	Before 3	Open	Before 3	Open	Before 3 After 8	Depends on Work Schedule	Depends on Work Schedule
Jack	After 5	After 5	After 8	After 5	After 5	Open	Open

Collaboration and Communication Plan

We plan to meet on Tuesdays, after Jack and Cole's class time (5:00-6:15), we will plan to have most meetings occur on Discord or Teams. However, if needed for the week, we will also have in-person meetings. This will occur in the library or any lab space we have available.

We will also meet based on Dr. Xie's availability as he will likely want to have weekly meetings as well.

Project Schedule and Task Planning

Project Name:	AI For Sensing																				
Report Date:	8/30/2025																				
						Milestone #1				*Milestone #2*				*Milestone #3*				C-Day			
Deliverable	**Tasks**	**Complete**	**Current Status Mem**	**Assigned To**		**09/02**	**09/09**	**09/16**	**09/23**	**09/30**	**10/07**	**10/14**	**10/21**	**10/28**	**11/04**	**11/11**	**11/18**	**11/25**	**12/02**		
Requirements	Meet with stakeholder(s) SH	0%		All		10	12														
	Define requirements	0%		All		10	10														
	Review requirements with SH	0%		Cole			10	15													
	Get sign off on requirements	0%		Jack				5	10												
Project design	Define tech required *	0%		Maddox					10	4											
	Hardware aquisition and familia	0%		Maddox, Jack					5	10	10										
	Sensor design	0%		Maddox, Jack							10	10									
	AI design	0%		Maddox, Jack				6	6	10	5	5									
	Develop working prototype	0%		Maddox, Jack					10	10	10	10									
	Test prototype	0%		All								5	10	5							
Development	Review prototype design	0%		Cole, Jack									8	5	10						
	Rework requirements	0%		Cole									8	10	12	5					
	Document updated design	0%		Cole, Jack												10	10				
	Test product	0%		All										8	8	5	20				
Final report	Presentation preparation	0%		All													15	10	10		
	Poster preparation	0%		Cole															10		
	Final report submission to D2L and project owner	0%		Cole															5		
			Total work hours		**402**	**20**	**32**	**26**	**41**	**34**	**35**	**30**	**26**	**28**	**30**	**20**	**45**	**10**	**25**		
	* formally define how you will develop this project including source code manag																				
Legend																					
Planned	402																				
Delayed																					
Number																					

Version Control Plan

We will use GitHub and our local machines for our version control. The most recent working version will be saved in the main repository on GitHub; we will back up other versions in separate repositories based on reaching certain milestones in the project. We will also post routine updated versions every week to ensure if there ever is a disaster scenario where our project is corrupted, we don't lose too much progress. Our Dev versions will be on our local machines and will be pushed to the GitHub on successful new additions.

Test Plan

1. Test Objectives
    - We are trying to verify that our Model can work with any correctly submitted mmWave radar data.
    - We are trying to verify that our Model's projections are accurate and comparable to the real room as well as LIDAR results when available.
2. Scope of Testing
    - In-scope: Features/modules you will test
    - We will test that the projection is outputted similarly to the legitimate space.
    - We will test that the model is able to output projections when data is uploaded in the correct method.
    - Out-of-scope: Features you will not test
    - We will not test how the model handles inappropriate data types.

3. Test Cases

- 1.0
    - Description: Our model should be trainable without throwing errors.
    - Expected Result: Our model throws no errors or exceptions
- 2.0
    - Description: Assess the accuracy of the model using F-score and chamfer distance.
    - Expected Result: Model has similar F-score and chamfer distances compared to the original UpPoinTr model/pipeline and other models.
- 3.0
    - Description: Other people should be able to access our model and recreate our results using our git page
    - Expected Result: Other people can easily access and set up our model to achieve similar results to ours.

4. Test Procedures
    - Steps to execute each test case
    - Example:
        - Step 1: Open app
        - Step 2: Enter username/password
        - Expected: Login successful

- 1.0
    - Step 1: Attempt to train model
    - Expected: A new model is created and trained.
- 2.0
    - Step 1: Train a model
    - Step 2: Record the accuracy scores outputted when model is trained.
    - Step 3: Compare these scores to original model and others.
    - Step 4: Inference the model to get an output and compare this to the original model.
    - Expected: Our performance will be worse but we should still be finding trends and patterns similar to the other models. Accuracy scores will be much worse but should still be compared.
- 3.0
    - Step 1: Clone our github repository.
    - Step 2: Install required dependencies.
    - Step 3: Train a custom model using our modified pipeline
    - Expected: Downloads and training are successful.

5. Test Environment

- Any machine learning model has intensive hardware requirements, and our system is no different. Our minimum recommended hardware is as follows; a multi-core processer such as an Intel Core i7, 64GB of RAM, a GPU with at least 12GB of VRAM such as NVIDIA GeForce RTX 3060. Our general recommended hardware is as follows; Intel Core i9, 128GB of RAM, multiple GPUs with at least 12 GB of VRAM. Our system will exist in its own Python Conda virtual environment. This environment will be running Python 3.11 and will utilize the PyTorch library among many others. Running a Linux distribution is optimal, however you are able to run the model on Windows via WSL.

6. Test Data

- The ColoRadar dataset has the necessary data to train and validate our models. It has both radar data, which is what we are training our model on, and lidar data which we will use as our ground truth. We will create a convolution neural network and train it on our dataset. From this, our model produces predictions of the environments based on the data that we trained the models on.

Summary

We believe that spiking neural networks are a realistic substitute for artificial neural networks when power consumption is a concern. We converted the up sampling module of the UpPoinTr pipeline to a spiking neural network and were able to produce similar results to the original spiking network.

Appendix

Bibliography

[1]BindsNET, “GitHub - BindsNET/bindsnet: Simulation of spiking neural networks (SNNs) using PyTorch.,” GitHub, Oct. 18, 2024. https://github.com/BindsNET/bindsnet (accessed Sep. 14, 2025).

[2]A. Stanojevic, Stanisław Woźniak, Guillaume Bellec, G. Cherubini, Angeliki Pantazi, and W. Gerstner, “High-performance deep spiking neural networks with 0.3 spikes per neuron,” Nature Communications, vol. 15, no. 1, Aug. 2024, doi: https://doi.org/10.1038/s41467-024-51110-5.

[3] Kramer, Andrew, Kyle Harlow, Christopher Williams, and Christoffer Heckman. “ColoRadar: The direct 3D millimeter wave radar dataset.” The International Journal of Robotics Research 41, no. 4 (2022): 351-360.